Document number: P2235R0
Audience: LEWG, SG1

Ville Voutilainen
On behalf of SFS (Finland)
2020-10-15

Disentangling schedulers and executors

Credits

Many many thanks to Tomasz Kamiński for a very high-quality technical review of this paper.

Abstract

This proposal proposes to simplify the design of schedulers and executors, providing a single conversion facility from an `executor` to a `scheduler`, removing the non-scheduler/sender/receiver facilities from `schedule` and `connect`. This proposal *also* removes the ability to `execute` on a sender.

Why?

Schedulers and executors are apples and oranges. Schedulers, senders, and receivers establish a generic protocol that is knit together so as to facilitate generic programming and algorithms that can operate within the framework of that protocol, transforming in various ways what senders and receivers do. Executors do not work with this protocol, because they do not provide two thirds of it; namely, they do not provide any means to ever invoke set_error or set_done.

Therefore treating an executor as a scheduler is akin to a lossy conversion. The conversion is sufficiently lossy that it would be unwise if it Just Happens without any indication in source code. That's why multiple reviewers have suggested that that conversion should be explicit. This proposal makes it explicit.

Once we accept that an executor shouldn't just be treated as a scheduler, we also realize that there's no point in providing support for treating a sender-of-void as an executor in the execution::execute() CPO. Such an operation is every bit as lossy as treating an executor as a scheduler. It should be likewise explicit, and there's no reason to provide it in the fundamental building blocks; a separate algorithm can be provided that connects a plain invocable to a sender.

In a slightly different vein, a scheduler or a sender should not be treated as an executor either. That's not like a lossy conversion, but it's like invoking a function with a wide contract, and then having it be overridden by a function with a narrow contract, and thus the caller expectations of a wide contract aren't met. The semantics of execute() are intentionally rather free-form. There is no generic protocol that executors conform to, but senders and receivers do have such a protocol, and schedulers *will* conform to it, causing naive users of execute on senders to shoot themselves in the foot. Furthermore, that protocol will intercept attempts to implement custom protocols solely in the operator() of the user-provided runnable; the handling of scheduling errors will happen in schedulers before that runnable is invoked, thwarting the attempts to implement something custom and breaking expectations for a custom protocol.

These apples and oranges don't mix. In either direction. Cross-pollination attempts should be explicit, visible, greppable, and done with utmost care.

How?

1. `schedule()` operates on schedulers only.
2. `execute()` is a customization point that operates on an executor only.
3. All senders&receivers-related operations, like `connect()`, operate on senders&receivers only.
4. There is a conversion function that takes an executor, and converts it to a scheduler. The working name for this function is `make_scheduler_from_executor()`, because that's what it does.
5. Separate algorithms, most likely and preferrably with names other than 'execute', can be provided in e.g. P1897 to allow straightforward fire-and-forget on schedulers or senders.

Rumination

The current design is complex, confusing, and error-prone

The current design allows execute() on anything, schedule() on anything, connect() on anything. But that makes no sense; execute() is okay as a one-way fire-and-forget mechanism when no particular error-handling semantics are expected, but schedule() on an executor tosses in the wind the `set_error/set_done` parts. Those parts are absolutely necessary for some senders&receivers use cases to work. Padding them in with operations that e.g. `terminate` shouldn't just happen willy-nilly. Otherwise it becomes impossible to reason about generic code, and to trust that senders are actual senders, receivers are actual receivers, and that the senders&receivers protocol actually works. As far as execute() goes, the story is similar. What we have right now will terminate() a program if there is a scheduling error. This becomes ever more likely when programs and applications using these facilities grow more complex; simple schedulers&senders&receivers combine into more complicated ones, increasing the chance that scheduling errors will not be just allocation failures, but something much more likely, such as i/o errors. Thus, simply executing on a sender that has terminate() called from its receivers' set_done/set_error is a massive footgun. It's also a massive footgun if we make executing on a sender actually work properly with the sender&receiver protocol, since transitioning to the executor world will again drop that protocol.

This proposal makes the design clearer. In order to make the jump from executors (which don't conform to any protocol) to senders&receivers, there is exactly one conversion operation that crosses that bridge (or rather jumps over the river when there is no bridge) Otherwise, the separate worlds are kept separate. Similarly, the jump from senders&receivers to executors doesn't introduce surprises when an execute function would do drastically different things depending on what it's invoked on. This avoids a vector<bool> problem that we currently have in the design of P0443. Any jumps back and forth between the worlds don't introduce surprises either.

Once in a senders&receivers world, all operations related to them work as expected. The cross-river jump mentioned before is easy to find, and is greppable. One doesn't need to look at all operations, including `schedule` and `connect` and wonder whether they're operating on a scheduler and a sender or perhaps an executor and an invocable.

Similarily, once in an executor world, the operations related to an executor work as expected. In order to have a protocol, or to deal with executor-specific means of handling scheduling errors, that needs to be programmed explicitly, instead of having a sender introduce a protocol where it's perhaps not expected or even desired.

It's still perfectly plausible to provide a function that queues a plain invocable to run as an error-ignoring receiver of a sender. But that's a separate named algorithm, not an overload of `connect`. It's still possible to indirectly call `schedule` on an executor, but that's `schedule(make_scheduler_from_executor(foo))`, not `schedule(foo)`.

The cross-concept bridging the current design attempts is not useful for programmers

We are going to see a fair amount of generic algorithms that operate on senders. These algorithms will provide decorators that change (sometimes extend, sometimes transform) what senders, receivers, and operation_states do. They will be lego-like building blocks that introduce a particular form of aspect-oriented programming into the world of senders and receivers.

Programmers, even library programmers, are expected to be mostly using such algorithms, rather than using things like `connect()` directly. These algorithms will be constrained to accept `schedulers, senders` and `receivers`. They will not be constrained to accept both `schedulers` and `executors` even if the cross-concept bridging that's currently in P0443 might make using them on `executors` well-formed.

And, again, code that uses executors will probably do so with the expectation that there is a particular custom protocol in play, or no protocol at all. Implicitly introducing the senders&receivers protocol into code that has the aforementioned expectation(s) can be a massively breaking change.

We need a clear, simple, and understandable story of what our concepts are and what they do. That means we shouldn't encourage every programmer to deal with a `scheduler_or_executor` concept that's basically a disjunction of the two concepts. Executors aren't schedulers, so let's not pretend that they are. For those who insist on treating an executor as a scheduler, we provide them with a very explicit conversion function. Executors are a perfectly reasonable family of types in and of themselves, types that provide a less strict protocol than schedulers do; that's fine, those types can be used by audiences who have no use for the senders-and-receivers protocol. But as long as those audiences don't have a use for the senders-and-receivers protocol, we make the conversion from an executor to a scheduler _deliberately_ ugly. And as long as those audiences don't have a use for the senders-and-receivers protocol, we don't inflict it on those audiences implicitly, either.

All that considered, is make_scheduler_from_executor fundamental for P0443?

That would be no. We could just as well do something like it separately in P1897. It's proposed here to provide a better overall view of the cross-concept picture, but doesn't strictly need to be in P0443.

What about bridging to the other direction? Why is there no make_executor_from_sender proposed here?

Such a facility is fraught with peril. A sender will report scheduling errors. For an executor, those errors have nowhere to go. Thus they would be unhandled errors, and would need to be intercepted and most likely call terminate(). What makes it worse is that you can take a sender, and apply an error-handling algorithm on top of it, which would otherwise intercept the set_error calls and do what that algorithm defines. But a hypothetical make_executor_from_sender wouldn't know that, so it would need to intercept set_error again, on top, and those intercepts would cause termination even if an algorithm that would make those intercepts unnecessary has already been applied.

All that considered, do we still need both concepts, executor and scheduler?

Yes, we do. Schedulers, senders, and receivers establish a scalable protocol that can deal with errors between task submission and callback of the invokable, and provide proper cleanup. However, if you need something completely different from that protocol, an executor may be a better fit. And to be able to write multiple different executors, they should still have a common API.

Changes to P0443

In 2.2.3.4 execution::execute, modify the second paragraph and remove the third bullet:

For some subexpressions e and f, let E be decltype((e)) and let F be decltype((f)). The expression execution::execute(e, f) is ill-formed if F does not model invocable, or if E does not model ~~either~~ executor ~~or sender~~. Otherwise, it is expression-equivalent to:

- `e.execute(f), if that expression is valid. If the function selected does not execute the function object f on the executor e, the program`
- `Otherwise, execute(e, f), if that expression is valid, with overload resolution performed in a context that includes the declaration`

  ```
  void execute();
  ```

  `and that does not include a declaration of execution::execute. If the function selected by overload resolution does not execute the funct`
- ~~`Otherwise, execution::submit(e, as_receiver<remove_cvref_t<F>, E>{forward<F>(f)}) if`~~
  - ~~`F is not an instance of as_invocable<R,E'> for some type R where E and E' name the same type ignoring cv and reference qualifiers, a`~~
  - ~~`invocable<remove_cvref_t<F>&> && sender_to<E, as_receiver<remove_cvref_t<F>, E>> is true`~~

  ~~`where as_receiver is some implementation-defined class template equivalent to:`~~

  ```
  template<class F, class&gT;
  struct as_receiver {
    F f_;
    void set_value() noexcept(is_nothrow_invocable_v<F&>) {
      invoke(f_);
    }
    template<class E>
    [[noreturn]] void set_error(E&&) noexcept {
      terminate();
    }
    void set_done() noexcept {}
  };
  ```

In 2.2.3.5 execution::connect, remove the third bullet:

- ~~Otherwise, as-operation{s, r}, if~~
  - ~~r is not an instance of as-receiver<F, S'> for some type F where S and S' name the same type ignoring cv and reference qualifiers, and~~
  - ~~receiver_of<R> && executor-of-impl<remove_cvref_t<S>, as-invocable<remove_cvref_t<R>, S>> is true,~~

  ~~where as-operation is an implementation-defined class equivalent to~~

```
~~template<class S, class R>~~
~~struct as-operation {~~
~~  remove_cvref_t<S> e_;~~
~~  remove_cvref_t<R> r_;~~
~~  void start() noexcept try {~~
~~    execution::execute(std::move(e_), as-invocable<remove_cvref_t<R>, S>{r_});~~
~~  } catch(...) {~~
~~    execution::set_error(std::move(r_), current_exception());~~
~~  }~~
~~};~~
```

  ~~and as-invocable is a class template equivalent to the following:~~

```
~~template<class R, class>~~
~~struct as-invocable {~~
~~  R* r_;~~
~~  explicit as-invocable(R& r) noexcept~~
~~    : r_(std::addressof(r)) {}~~
~~  as-invocable(as-invocable && other) noexcept~~
~~    : r_(std::exchange(other.r_, nullptr)) {}~~
~~  ~as-invocable() {~~
~~    if(r_)~~
~~      execution::set_done(std::move(*r_));~~
~~  }~~
~~  void operator()() & noexcept try {~~
~~    execution::set_value(std::move(*r_));~~
~~    r_ = nullptr;~~
~~  } catch(...) {~~
~~    execution::set_error(std::move(*r_), current_exception());~~
~~    r_ = nullptr;~~
~~  }~~
~~};~~
```

- Otherwise, execution::connect(s, r) is ill-formed.

In 2.2.3.8 execution::schedule, remove the third bullet:

- ~~Otherwise, as-sender<remove_cvref_t<S>>{s} if S satisfies executor, where as-sender is an implementation-defined class template equivalent to~~

```
~~template<class E>~~
~~struct as-sender {~~
~~private:~~
~~  E ex_;~~
~~public:~~
~~  template<template<class...> class Tuple, template<class...> class Variant>~~
~~    using value_types = Variant<Tuple<>>;~~
~~  template<template<class...> class Variant>~~
~~    using error_types = Variant<std::exception_ptr>;~~
~~  static constexpr bool sends_done = true;~~

~~  explicit as-sender(E e) noexcept~~
~~    : ex_((E&&) e) {}~~
~~  template<class R>~~
~~    requires receiver_of<R>~~
~~  connect_result_t<E, R> connect(R&& r) && {~~
~~    return execution::connect((E&&) ex_, (R&&) r);~~
~~  }~~
~~  template<class R>~~
~~    requires receiver_of<R>~~
~~  connect_result_t<const E &, R> connect(R&& r) const & {~~
~~    return execution::connect(ex_, (R&&) r);~~
~~  }~~
~~};~~
```

- Otherwise, execution::schedule(s) is ill-formed.

After 2.2.3, add a new section:

> 2.x execution::make_scheduler_from_executor
>
> The behavior of a program that adds specializations for `make_scheduler_from_executor` is undefined.
>
> `template <class E> scheduler_auto make_scheduler_from_executor(E&& executor);`
>
> Constraints: remove_cvref_t<E> satisfies execution::executor.
>
> Preconditions: remove_cvref_t<E> models execution::executor.
>
> Returns:
>
> a `scheduler`, so that calling `execution::schedule(s)` on that `scheduler` `s` returned from `make_scheduler_from_executor` is expression-equivalent to

as-sender<remove_cvref_t<S>>{executor}, where as-sender is an implementation-defined class template equivalent to

```
  template<class E>
  struct as-sender {
  private:
    E ex_;
  public:
    template<template<class...> class Tuple, template<class...> class Variant>
      using value_types = Variant<Tuple<>>;
    template<template<class...> class Variant>
      using error_types = Variant<std::exception_ptr>;
    static constexpr bool sends_done = true;

    explicit as-sender(E e) noexcept
      : ex_((E&&) e) {}
    template<class R>
      requires receiver_of<R>
    auto connect(R&& r) && {
      return as-operation<E, remove_cvref_t<R>>{(E&&)ex_, (R&&) r};
    }
    template<class R>
      requires receiver_of<R>
    auto connect(R&& r) const & {
      return as-operation<E, remove_cvref_t<R>>{ex_, (R&&) r};
    }
  };
```

where as-operation is an implementation-defined class template equivalent to

```
    template <class E, class R>
    struct as-operation {
    E e_;
    R r_;
    void start() noexcept try {
      execution::execute(std::move(e_), as-invocable<R, E>{r_});
    } catch(...) {
      execution::set_error(std::move(r_), current_exception());
    }
  };
```

and as-invocable is an implementation-defined class template equivalent to the following:

```
  template<class R, class>
  struct as-invocable {
    R* r_;
    explicit as-invocable(R& r) noexcept
      : r_(std::addressof(r)) {}
    as-invocable(as-invocable && other) noexcept
      : r_(std::exchange(other.r_, nullptr)) {}
    ~as-invocable() {
      if(r_)
        execution::set_done(std::move(*r_));
    }
    void operator()() & noexcept try {
      execution::set_value(std::move(*r_));
      r_ = nullptr;
    } catch(...) {
      execution::set_error(std::move(*r_), current_exception());
      r_ = nullptr;
    }
  };
```

What This Buys

The user can no longer pass an executor to `schedule()` or `connect()`, and have the code be well-formed, but with broken runtime semantics. Such code is extremely likely to be a mistake.

The user can no longer pass a sender to `execute()`, and have the code be well-formed, but with broken runtime semantics. Such code is extremely likely to be a mistake.